Chap11_File_Access | Directory | File_System | Mount | Protection | Sharing

Chap11_File_Access | Directory | File_System | Mount | Protection | Sharing

Access Methods

- Two ways:
  - Sequential Access (notepad, editors etc)
  - Direct Access (based on direct disk block access)
- Another types involve indexing
  - Single layered
  - Multilayered
- **Direct Access**
  - Also called relative access
  - File made up of fixed length logical records that allow programs to read and write records rapidly in no particular order.
  - Based on **disk model of a file,** since disks allow random access to any file block
  - File viewed as a numbered sequence of blocks and records
  - **No restriction on order of reading and writing**
  - **Method:**
    - **the file operations** must be modified to **include the block number as a parameter.**
    - **Relative block number** is the index relative to beginning of file (JAVA errors line number are relative)
  - Easier to read, writem delete a record because logical records are of fixed size.

- Not all OS supports sequential as well as direct access, both.
- Simluating a **direct access on a sequential access file** however could be extremely inefficient and clumsy
-

sequential access	implementation for direct access

reset	cp = 0;
read_next	read cp; cp = cp + 1;
write_next	write cp; cp = cp + 1;

Figure 11.5 Simulation of sequential access on a direct-access file.

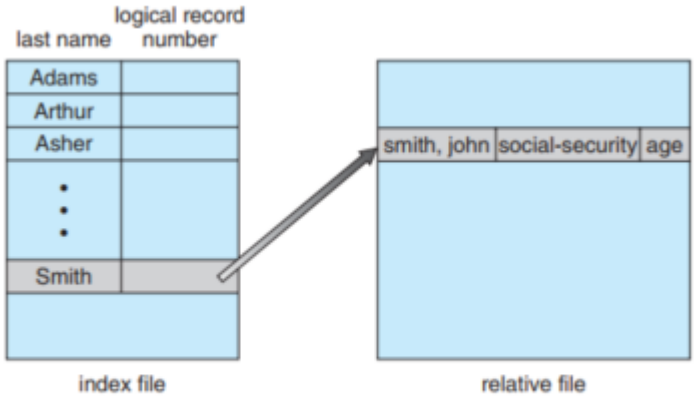
- **Other access methods**
  - Indexed containing pointers to the block on disk
  - Requires storage of index also,
  - Single index file may be too large, so two layer indexing might be used
  - **Binary search can be used after sorting the blocks according to some data value(unique)**
  - 

Figure 11.6 Example of index and relative files.

Directory and Disk Structure

- **Volume is** an entity containing a file system, which may be on **more than one disks, linked together using RAID sets.**
- Inforamtion about files on the file system is kept in **directory or volume table of contents,** such as name, location, etc.

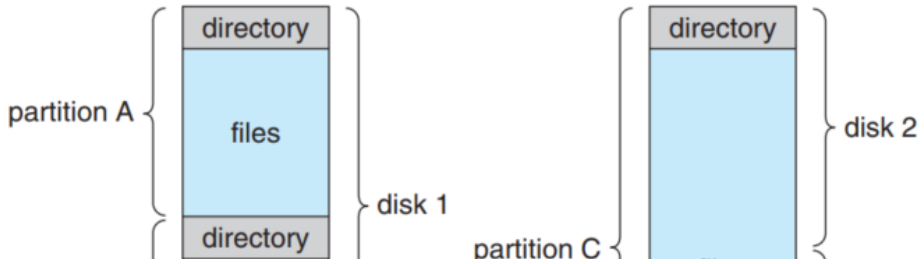


Figure 11.7 A typical file-system organization.

/	ufs
/devices	devfs
/dev	dev
/system/contract	ctfs
/proc	proc
/etc/mnttab	mntfs
/etc/svc/volatile	tmpfs
/system/object	objfs
/lib/libc.so.1	lofs
/dev/fd	fd
/var	ufs
/tmp	tmpfs
/var/run	tmpfs
/opt	ufs
/zpbge	zfs
/zpbge/backup	zfs
/export/home	zfs
/var/mail	zfs
/var/spool/mqueue	zfs
/zpbg	zfs
/zpbg/zones	zfs

Figure 11.8 Solaris file systems.

- Storage Structure
  - File system types
    1. **tmpfs**—a "temporary" file system that is created in volatile main memory and has its contents erased if the system reboots or crashes
    2. **objfs—a** "virtual" file system (essentially an interface to the kernel that looks like a file system) that gives debuggers access to kernel symbols
    3. **ctfs—**a virtual file system that maintains "contract" information to manage which processes start when the system boots and must continue to unduring operation
    4. **lofs**—a "loop back" file system that allows one file system to be accessed in place of another one
    5. **procfs**—a virtual file system that presents information on all processes as a file system
    6. **ufs, zfs**—general-purpose file systems
  - Directory
    - The organization must allow us to insert entries, to delete entries, to search for a named entry, and to list all the entries in the directory.
    - Operations performed on a directory structure
      1. • **Search for a file**. We need to be able to search a directory structure to find the entry for a particular file. Since files have symbolic names, and similar names may indicate a relationship among files, we may want to be able to find all files whose names match a particular pattern.
      2. • **Create a file**. New files need to be created and added to the directory

2. • **Create a file**. New files need to be created and added to the directory.
3. • **Delete** a file. When a file is no longer needed, we want to be able to remove it from the directory.
4. • **List a directory**. We need to be able to list the files in a directory and the contents of the directory entry for each file in the list.
5. • **Rename a file.** Because the name of a file represents its contents to its users, we must be able to change the name when the contents or use of the file changes. Renaming a file may also allow its position within the directory structure to be changed.
6. • **Traverse the file system**. We m**ay wish to access every directory and every file within a directory st**ructure. For reliability, it is a good idea to save the contents and structure of the entire file system at regular intervals. Often, we do this by copying all files to magnetic tape. This technique provides a backup copy in case of system failure. In addition, if a file is no longer in use, the file can be copied to tape and the disk space of that file released for reuse by another file

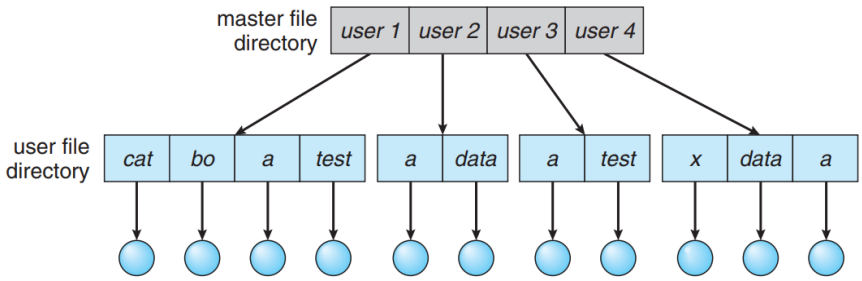
- **Logical structure of directory**
  - **Single Level directory**
    - Files should have unique names,
    - Not good for large number of files.
  - **Two level directory**
    - For multiple users, multiple users can have a file with the same name
    - Needed when users need to collaborate and work on both user's space, having fie with same name
    - **UFD - User file directory**
    - **MFD - Master File dorectory containg user names and location on disk**
    - The MFD is indexed by user name or account number, and each entry points to the UFD for that user
    - 

      Figure 11.10 Two-level directory structure.

    - MFD usage is restricted to system admins
    - a user name and a file name define a path name. Every file in the system has a path name. To name a file uniquely, a user must know the path name of the file desired.
    - **File Naming:**
      - Windows a volume is specified by a letter followed by a colon.
      - In VMS, for instance, the file login.com might be specified as:
        - u:[sst.jdeck]login.com;1,
        - where u is the name of the volume,

        - sst is the name of the directory,
        - jdeck is the name of the subdirectory, and
        - 1 is the version number.
    - Other systems—such as UNIX and Linux—simply treat the volume name as part of the directory name.
        - The first name given is that of the volume, and the rest is the directory and file.
        - For instance, /u/pbg/test might specify volume u, directory pbg, and file test.
- Searching for system files
    - System files are only stored at one location, if one needs them, either create a copy of it for herself in her own **UFD**
    - But this would **waste too much space for each user with her own system files**
    - Solution is to Have a **USER_0** who has system files in her UFD
    - **Search path:** the **sequence of directories** when a **file is named**
    - Search path is used widely in Linux and Windows.
    - Search path for system files by any non-admin user is
        - First search her own UFD
        - IF (found) then Okay,
        - **ELSE search for USER_0 UFD**

- **Tree Level directory**
    - Extension to the previous approach
    - User can access other user's file too, even system files too, no need for special user_0
    - A root directory, and each file has a unique path name
    - More flexible, lets the user create directories and subdirectories, which may contain files of same name
    - In this approach, **a tree is just another file but with the first bit set (LINUX drwx--x--x is a directory, -rwx--x--x is a FILE srqx--x--x is a SETUID binary**
    - When the user logs in, the system searches for an **accounting file** which **contains the pointer to user's initial directory.** All the subprocesses have this directory as the current directory.
    - Path names can be **relative or absolute**
    - **Situation when a directory needs to be deleted**
        - **Two approaches:**
            - **IF_directory (is_empty)**
                - its entry in the directory that contains **it can simply be deleted.**
            - **ELSE**

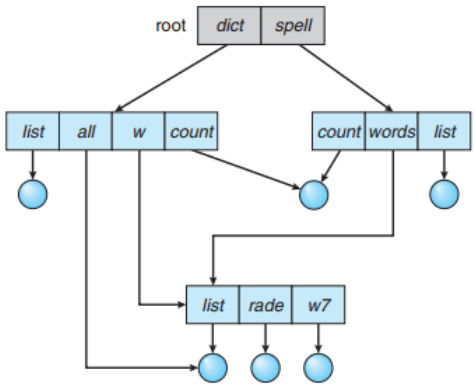
        - Two approaches
            - **NOT DELETE unless empty**
                - Thus, to delete a directory, the user must first delete all the files in that directory.
                - If any sub-directories exist, this procedure must be applied recursively to them, so that they can be deleted also.
                - This approach can result in a **substantial amount of work**.
            - UNIX **rm command**, is to **provide an option**:
                - when a request is made to delete a directory, all that directory's files and subdirectories are also to be deleted.
        - Either approach is fairly easy to implement; the choice is one of policy.
        - The latter policy is more convenient, but it is also more dangerous, because an entire directory structure can be removed with one command.
        - If that command is issued in error, a large number of files and directories will need to be restored
- **Acyclic-Graph Directories**
    - Graph with no cycles
    - Allows directory to share sub-directories
    - Not the same as copying the file.
    - With shared file, only **one actual file exists,** any changes made by one person are immediately visible to the other
    - 

      Figure 11.12 Acyclic-graph directory structure.
    - Some common ways to implement it :
        1. **LINK** in UNIX systems, pointer to another file
            - may be implemented as **an absolute or relative path name**
            - OS ignores these links when traversing directory trees to preserve the acyclic structure of system
        2. Duplicate all information about them in both sharing directories
            - Duplicate directory entries, make the original and copy indistinguishable

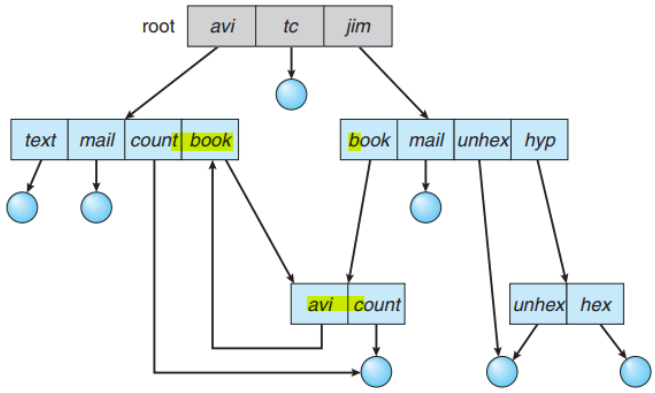
        - UNDERSTAND THIS, NOT CLEAR
    - More flexible than tree but more complex,
    - **DISADVANTAGE**S:
        - A file may now have **multiple absolute path** names, to find a file, traversing shared structures more than once is a problem (SOLUTION do not traverse shared objects, **NEed of General Graph Directory**)
        - **Deletion,**
            - removing the file straightaway, would **leave dangling pointers.**
                - **If** the space for the actual file is reused by another file, these dangling ponters may point into the middle of other files
            - **SYMBOLIC links** systems, like WINDOWS and **UNIX (Soft links)**, do this
    - Solution to deletion problem
        - Preserve the file until all references to it are deleted, for this, the actual file needs to have a count of #pointers to it, when it becomes 0, the file can be deleted
        - **UNIX Hard links are an example**
- **General Graph directory**
    - Built to tackle the problem of cycling in acyclic graphs
    - 

Figure 11.13 General graph directory.
    - See the cycle from directory avi to book to avi.
    - Also using a reference counter would be useless, if a directory is pointing to itself (self referencing)
    - Thus we use **Garbage collection**
        - Garbage collection involves **traversing the entire file system, marking everything that can be accessed.**
        - Then, **a second pass collects everything that is not marked onto a list of free space**. (A similar marking procedure can be used to ensure that a traversal or search will cover everything in the file system once and only once.)
        - Garbage collection for a d**isk-based file system**, however, is **extremely time**

consuming and is thus seldom attempted.
        - Garbage collection is necessary only because of possible cycles in the graph.
        - Thus, an acyclic-graph structure is much easier to work with.
    - **A simpler algorithm** in the special case of directories and links is to **bypass links during directory traversal**

File System Mounting

- A file system must be mounted before it can be available to processes on the system.
- OS verifies the validity of file system, whether it can read it or not, like windows can not read UNIX ext4 file system.

File Sharing

- **Local File Systems**
    - Using access modifiers of file, drwxrwxrwx, etc in linux, Windows uses **icacls user etc.**

- **Remote File systems**
    - First implemented - FTP
    - Second impl. - Distributed file systems. (Search for examples), in which directories are visible from a local machine (**SCP protocol)**
    - Third impl. - WWW
    - Cloud Computing
    - FTP can be used for **anonymous as well as authenticated file exchange,**
    - **WWW almost exclusively used anonymous file exchange**, in which a user can transfer files

without having an account on the remote system.

-
- **Client Server Model**
  - Server has the file, clients wants to access it.
  - **Client identification is a difficulty,**
    - Usage of same encryption mechanism required by both parties
    - **IP address, Network Name can be easily spoofed**
  - UNIX systems uses NFS (Network File systems)
    - Authentication via the **client networking information**
    - **User's ID on the server and client must match** to set up connection
      - A user with ID 1000 on the client should have ID 1000 on server too.
    - After gaining access, the file operations requests are sent on ebhalf of the user across the network **via the DFS protocol**
- **Distributed Information Systems**
  - **11.5.2.2 Distributed Information Systems**

    To make client–server systems easier to manage, distributed information systems, also known as distributed naming services, provide unified access to the information needed for remote computing. The domain name system (DNS) provides host-name-to-network-address translations for the entire Internet. Before DNS became widespread, files containing the same information were sent via e-mail or `ftp` between all networked hosts. Obviously, this

    Unfortunately, it uses unsecure authentication methods, including sending user passwords unencrypted (in clear text) and identifying hosts by IP address. Sun's NIS+ was a much more secure replacement for NIS but was much more complicated and was not widely adopted.

    In the case of Microsoft's common Internet file system (CIFS), network information is used in conjunction with user authentication (user name and password) to create a network login that the server uses to decide whether to allow or deny access to a requested file system. For this authentication to be valid, the user names must match from machine to machine (as with NFS). Microsoft uses active directory as a distributed naming structure to provide a single name space for users. Once established, the distributed naming facility is used by all clients and servers to authenticate users.

    The industry is moving toward use of the lightweight directory-access protocol (LDAP) as a secure distributed naming mechanism. In fact, active directory is based on LDAP. Oracle Solaris and most other major operating systems include LDAP and allow it to be employed for user authentication as well as system-wide retrieval of information, such as availability of printers. Conceivably, one distributed LDAP directory could be used by an organization to store all user and resource information for all the organization's computers. The result would be secure single sign-on for users, who would enter their authentication information once for access to all computers within the
- **Failure Modes**
  - Local file systems can fail for a variety of reasons, including failure of the disk containing the file system, corruption of the directory structure or other disk-management information (collectively called metadata), disk-controller failure, cable failure, and host-adapter failure. User or system-administrator failure can also cause files to be lost or entire directories or volumes to be

deleted. Many of these failures will cause a host to crash and an error condition to be displayed, and human intervention will be required to repair the damage.

Remote file systems have even more failure modes. Because of the complexity of network systems and the required interactions between remote machines, many more problems can interfere with the proper operation of remote file systems. In the case of networks, the network can be interrupted between two hosts. Such interruptions can result from hardware failure, poor hardware configuration, or networking implementation issues. Although some networks have built-in resiliency, including multiple paths between hosts, many do not. Any single failure can thus interrupt the flow of DFS commands.

Consider a client in the midst of using a remote file system. It has files open from the remote host; among other activities, it may be performing directory lookups to open files, reading or writing data to files, and closing files. Now consider a partitioning of the network, a crash of the server, or even a scheduled shutdown of the server. Suddenly, the remote file system is no longer reachable. This scenario is rather common, so it would not be appropriate for the client system to act as it would if a local file system were lost. Rather, the system can either terminate all operations to the lost server or delay operations until the

- server is again reachable. These failure semantics are defined and implemented as part of the remote-file-system protocol. Termination of all operations can result in users' losing data—and patience. Thus, most DFS protocols either enforce or allow delaying of file-system operations to remote hosts, with the hope that the remote host will become available again.

  To implement this kind of recovery from failure, some kind of state information may be maintained on both the client and the server. If both server and client maintain knowledge of their current activities and open files, then they can seamlessly recover from a failure. In the situation where the server crashes but must recognize that it has remotely mounted exported file systems and opened files, NFS takes a simple approach, implementing a stateless DFS. In essence, it assumes that a client request for a file read or write would not have occurred unless the file system had been remotely mounted and the file had been previously open. The NFS protocol carries all the information needed to locate the appropriate file and perform the requested operation. Similarly, it does not track which clients have the exported volumes mounted, again assuming that if a request comes in, it must be legitimate. While this stateless approach makes NFS resilient and rather easy to implement, it also makes it unsecure. For example, forged read or write requests could be allowed by an NFS server. These issues are addressed in the industry standard NFS Version 4, in which NFS is made stateful to improve its security, performance, and functionality.

- **Consistency Semantics**
  - Consistency semantics are directly related to the process synchronization algorithms of Chapter 5.
  - However, the complex algorithms of that chapter tend not to be implemented in the case of file I/O because of the great latencies and slow transfer rates of disks and networks
  - UNIX Semantics
    - The UNIX file system (Chapter 17) uses the following consistency semantics:
      1. Writes t**o an open file by a user are visible immediately to other users who have this** file open.
      2. One mode of sharing **allows users to share the pointer of current location into the file.** Thus, the **advancing of the pointer by one user affects all sharing users. Here, a file has a single image that interleaves all accesses, re**gardless of their origin
  - Session Semantics(Evernote)
    - The Andrew file system (OpenAFS) uses the following consistency semantics:
      - Writes to an open file by a u**ser are not visible immediately to other** users that have the same file open.

- Once a file is closed, the ch**anges made to it are visible only in sessions starting later**. Already open instances of the file do not reflect these changes

- Immutable-Shared-Files Semantic
  - Once a file is **declared as shared by its creator, it cannot be modified.**
  - An immutable file has **two key proper**ties:
    1. its **name may not be reused,** and
    2. **its contents may not be altere**d.
  - Thus, the name of an immutable file signifies that the contents of the file are fixed.
  - The implementatio**n of these semantics in a distributed system** (Chapter 17) is **simple, because the sharing is disciplined (read-only).**

Protection

- **Types of Access**
  - **Read**. Read from the file.
  - **Write**. Write or rewrite the file.
  - **Execute**. Load the file into memory and execute it.
  - **Append**. Write new information at the end of the file.
  - **Delete**. Delete the file and free its space for possible reuse.
  - **List**. List the name and attributes of the file.

Other operations, such as renaming, copying, and editing the file, may also be controlled. For many systems, however, these higher-level functions may be implemented by a system program that makes lower-level system calls. Protection is provided at only the lower level. For instance, copying a file may be implemented simply by a sequence of read requests. In this case, a user with read access can also cause the file to be copied, printed, and so on.

Many protection mechanisms have been proposed. Each has advantages and disadvantages and must be appropriate for its intended application. A

- **Access Control**
  - The most common approach to the protection problem is to m**ake access dependent on the identity of the user.**
  - Different users may need different types of access to a file or directory. The most general scheme to implement identity dependent access is to associate with each file and directory an a**ccess-control list** (ACL) specifying user names and the types of access allowed for each user.

- This approach has the advantage of enabling complex access methodologies. The main problem with access lists is their length. If we want to allow everyone to read a file, we must list all users with read access. This technique has two undesirable consequences:

  - Constructing such a list may be a tedious and unrewarding task, especially if we do not know in advance the list of users in the system.
  - The directory entry, previously of fixed size, now must be of variable size, resulting in more complicated space management.

  These problems can be resolved by use of a condensed version of the access list.

  To condense the length of the access-control list, many systems recognize three classifications of users in connection with each file:

  - **Owner**. The user who created the file is the owner.
  - **Group**. A set of users who are sharing the file and need similar access is a group, or work group.
  - **Universe**. All other users in the system constitute the universe.

- One difficulty in combining approaches comes in the user interface. Users must be able to tell when the optional ACL permissions are set on a file. In the Solaris example, a "+" is appended to the regular permissions, as in:

  ```
  19 -rw-r--r--+ 1 jim staff 130 May 25 22:13 file1
  ```

  A separate set of commands, `setfacl` and `getfacl`, is used to manage the ACLs.

- Similar to **icacls in Windows, setfacl and getfacl**

- Another difficulty is assigning precedence when permission and ACLs conflict. For example, if Joe is in a file's group, which has read permission, but the file has an ACL granting Joe read and write permission, should a write by Joe be granted or denied? Solaris gives ACLs precedence (as they are more fine-grained and are not assigned by default). This follows the general rule that specificity should have priority.

  ### 11.6.3 Other Protection Approaches

  Another approach to the protection problem is to associate a password with each file. Just as access to the computer system is often controlled by a

- Thus, listing the contents of a directory must be a protected operation. Similarly, if a path name refers to a file in a directory, the user must be allowed access to both the directory and the file. In systems where files may have numerous path names (such as acyclic and general graphs), a given user may have different access rights to a particular file, depending on the path name used.

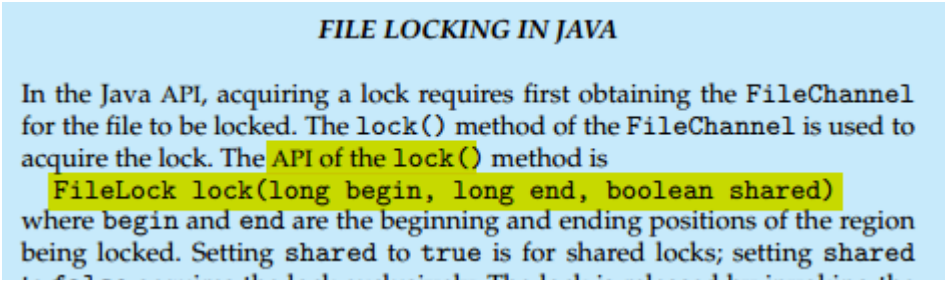
Chap11_File_System_Interface

- The operating system abstracts from the physical properties of its storage devices to define a logical storage unit, the file.

A file is the smallest allotment of logical secondary storage; that is, data cannot be written to

- A file is the smallest allotment of logical secondary storage; that is, data cannot be written to secondary storage unless they are within a file.
  - A **text fi**le is a sequence of characters organized into lines (and possibly pages).
  - A **source** file is a sequence of functions, each of which is further organized as declarations followed by executable statements.
  - An **executable file i**s a series of code sections that the loader can bring into memory and execute.
- The information about all files is kept in the directory structure, which also resides on secondary storage.
- Because directories, like files, must be nonvolatile, they must be stored on the device and brought into memory piecemeal, as needed
- ## File operations
  - ### File-ops
    - **Create**
      - **Two steps:**
        - Space in the file system must be found for the file
        - An entry fot the new file must be made in the directory
    - **Delete**
    - **Read**
      - Again, the directory is searched for the associated entry, and the system needs to keep a read pointer to the location in the file where the next read is to take place.
      - Once th**e read has taken place, the read pointer** is updated.
      - Because a process is usually either reading from or writing to a file, the current operation locatio**n can be kept as a per-process currentfile-position p**ointer.
      - Both the **read and write operations use this same pointer**, saving space and reducing system complexity
    - **Write**
      - To write, a system call specifying name of file and inforation to be written.
      - System must a **write pointer** to the location in the file where the next write is to take place
    - **Truncate**
      - Rather than forcing the user to delete the file and then recreate it, this function allows all attributes to remain unchanged—except
      - for file length—but lets the file be reset to length zero and its file space
      - released.
    - **Repositioning**
      - The directory is searched for the appropriate entry, and t**he current-file-position**

pointer is repositioned to a given value.
    - Repositioning within a file need not involve any actual I/O.
    - This file operation is also known as a file seek.

- The OS keeps a table, called the **open file table,** containing info about all open files. FYI, the **create() and delete()** are system calls that work with **closed rather than open files**
- Situation where several processes may open the file simultaneously.
    - Two levels of internal tables: **per process table and system wide table**
    - **System wide table has process independent information.**
    - Each entry in per process in turn points to system wide open file table
    - **Open count** associated with each file to indicate how many processes have the file open, each **close()** decreases this open count, zero means file entry removed from poen file table
- Information associated with open file:
    - **File pointer :**last read write location as a current file pointer, unique for each process operating, kept separate fro the on -disk file attributes
    - **FIle open count**
    - **Disk locaiton**
    - **Access rights**
- ## File locks
    - File locks are useful for files that are sha**red by several processes, similar to reader-writer locks**
    - A **shared lock** is akin to a reader lock in that several processes can acquire the lock concurrently.
    - An **exclusive lock** behaves like a writer lock only one process at a time can acquire such a lock
    - Locks can be **mandatory or advisory**
        - If a lock is mandatory, then once a process acquires an exclusive lock, the operating system will prevent any other process from accessing the locked file
        - if the lo**cking scheme is mandatory, the operating system ensures locking integrity.**
        - For **advisory locking, it is up to software developers** to ensure that locks are appropriately acquired and released.
    - 

FILE LOCKING IN JAVA

In the Java API, acquiring a lock requires first obtaining the `FileChannel` for the file to be locked. The `lock()` method of the `FileChannel` is used to acquire the lock. The API of the `lock()` method is

```
FileLock lock(long begin, long end, boolean shared)
```

where `begin` and `end` are the beginning and ending positions of the region being locked. Setting `shared` to `true` is for shared locks; setting `shared`

to `false` acquires the lock exclusively. The lock is released by invoking the `release()` of the `FileLock` returned by the `lock()` operation.

The program in Figure 11.2 illustrates file locking in Java. This program acquires two locks on the file `file.txt`. The first half of the file is acquired as an exclusive lock; the lock for the second half is a shared lock.

- ***FILE LOCKING IN JAVA (Continued)***

```
import java.io.*;
import java.nio.channels.*;

public class LockingExample {
  public static final boolean EXCLUSIVE = false;
  public static final boolean SHARED = true;

  public static void main(String args[]) throws IOException {
    FileLock sharedLock = null;
    FileLock exclusiveLock = null;

    try {
      RandomAccessFile raf = new RandomAccessFile("file.txt","rw");

      // get the channel for the file
      FileChannel ch = raf.getChannel();

      // this locks the first half of the file - exclusive
      exclusiveLock = ch.lock(0, raf.length()/2, EXCLUSIVE);

      /** Now modify the data . . . */

      // release the lock
      exclusiveLock.release();
```